Name:	Swathi Tathireddy
Email:	Tathireddyswathi190902@gmail.com
Reference No:	CSN-OC-OL-26408
Registration No.	CSN-OC-OL-26408
Domain:	Cyber Security
Project Level:	Advanced Level — OverTheWire Bandit Challenge (Levels 0–20)

S.No.	Project Description	Page No.
1	OverTheWire Bandit Challenge — solved Levels 0 to 20. For each level: the objective, exact commands used, reasoning behind the solution, the password obtained, and a screenshot of the output.	2
2	**Blog** **Title:** What I Learned Hacking Through OverTheWire Bandit (Levels 0–20)	18

1. Introduction

OverTheWire Bandit is a free, legal, beginner-friendly 'wargame' designed to teach the basics of Linux, the command line, and security concepts. It is made up of sequential levels. Each level is a small puzzle; solving it reveals a password that is used to log into the next level. This report documents my work through Levels 0 to 20, showing the objective of each level, the exact commands I used, my reasoning, the password I obtained, and a screenshot of the output as proof.

2. Environment & Tools

• **Platform:** OverTheWire Bandit — bandit.labs.overthewire.org, SSH port 2220

• **Access method:** SSH from a local terminal (Windows PowerShell / macOS Terminal / Linux shell)

• **Key commands/tools used:** ssh, ls, cat, file, find, grep, sort, uniq, strings, base64, tr, xxd, diff, nmap, nc (netcat), openssl, setuid binaries

3. Level-by-Level Walkthrough

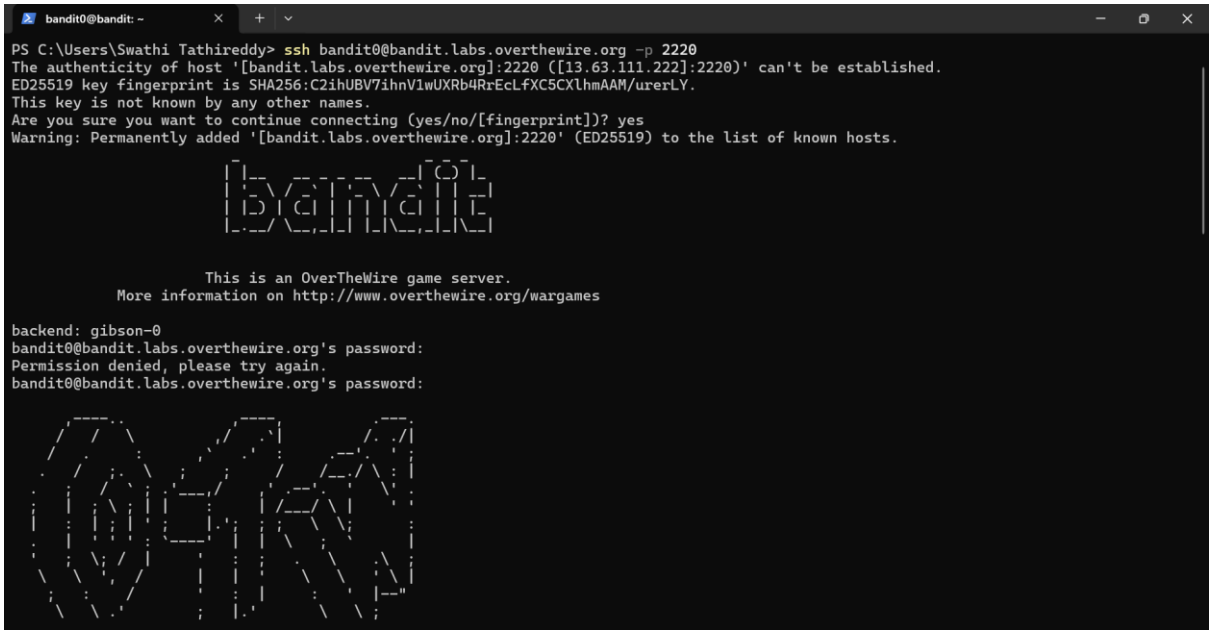
Level 0 — Logging In

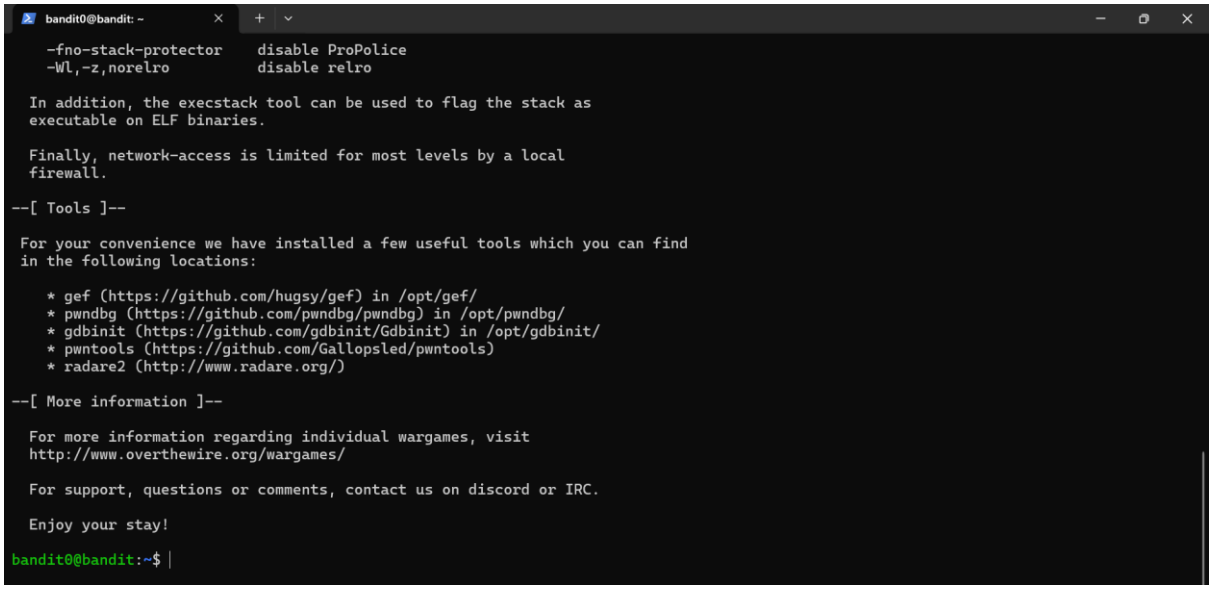
Objective: Connect to the Bandit game server over SSH using the starting credentials provided on the OverTheWire website.

Commands used:

```
ssh bandit0@bandit.labs.overthewire.org -p 2220
# password when prompted:  bandit0
```

Reasoning / Explanation: SSH (Secure Shell) is a safe way to log into a remote computer's command line. Port 2220 is where the Bandit server listens. Note: the password will NOT appear on screen as you type it — this is a normal Linux security behaviour, so just type it and press Enter.





Level 0 → Level 1

Objective: The password for the next level is stored in a file named 'readme' in the home directory.

Commands used:

```
ls
cat readme
```

Reasoning / Explanation: 'ls' lists the files in the current folder — you will see 'readme'. 'cat' prints a file's contents to the screen, revealing the password.

Password obtained: __6y2kwnwK6grgvwvpvLaa2T1cpFEKOhNR_____



Level 1 → Level 2

Objective: Read a file whose name is a single dash character ('-').

Commands used:

```
cat ./-
```

Reasoning / Explanation: A filename that starts with '-' is mistaken by the system for a command option. Prefixing it with './' forces the shell to treat it as a file path in the current directory.

Password obtained: _____PK8fYLZg2hnHSz83plBL1iEPKdD3QToB_____





Level 2 → Level 3

Objective: Read a file whose name contains spaces ('spaces in this filename').

Commands used:

```
cat "spaces in this filename"
# alternative: cat spaces\ in\ this\ filename
```

Reasoning / Explanation: Spaces normally split a command into separate words. Wrapping the name in double quotes (or escaping each space with a backslash) keeps it as one single filename.

Password obtained: 7ZZ2LFrykP2zEyvBl4m3clcL7tGYJPME





Level 3 → Level 4

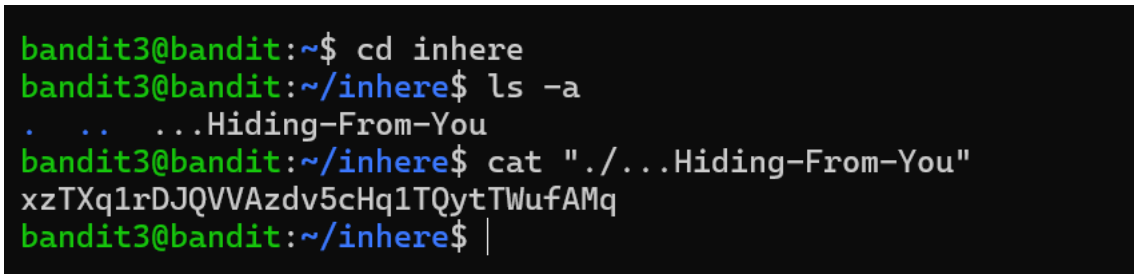
Objective: Find a hidden file inside the 'inhere' directory.

Commands used:

```
cd inhere
ls -a
cat ./...Hiding-From-You
```

Reasoning / Explanation: Files whose names begin with a dot ('.') are hidden by default. 'ls -a' shows all files including hidden ones. We then read the hidden file (name may differ slightly — use what 'ls -a' shows).

Password obtained: xzTXq1rDJQVVAzdv5cHq1TQytTWufAMq



Level 4 → Level 5

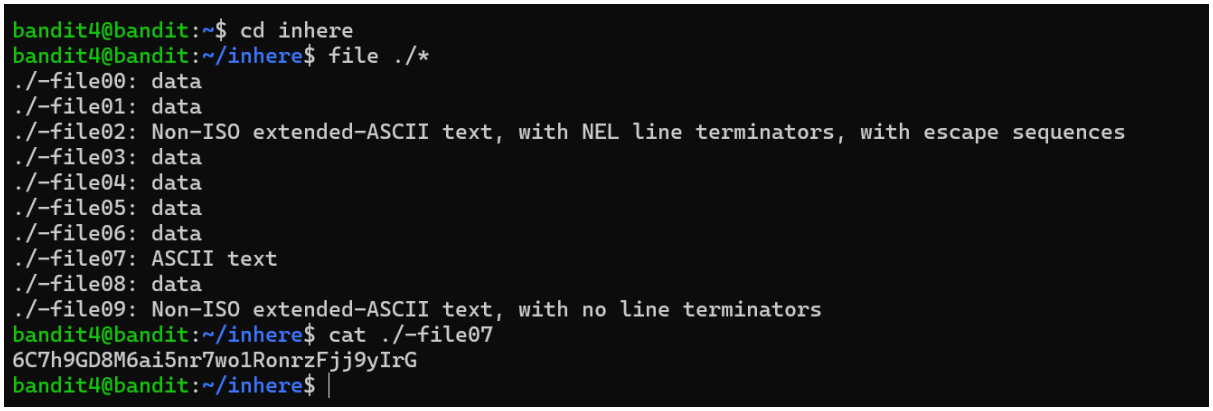
Objective: Find the only human-readable file among several files inside 'inhere'.

Commands used:

```
cd inhere
file ./*
cat ./-file07   # read the file reported as ASCII text
```

Reasoning / Explanation: 'file' reports the type of each file. Most are data/binary; only one is 'ASCII text'. Read that one. The exact name may vary, so use the one 'file' marks as text.

Password obtained: 6C7h9GD8M6ai5nr7wo1RonrzFjj9yIrG



Level 5 → Level 6

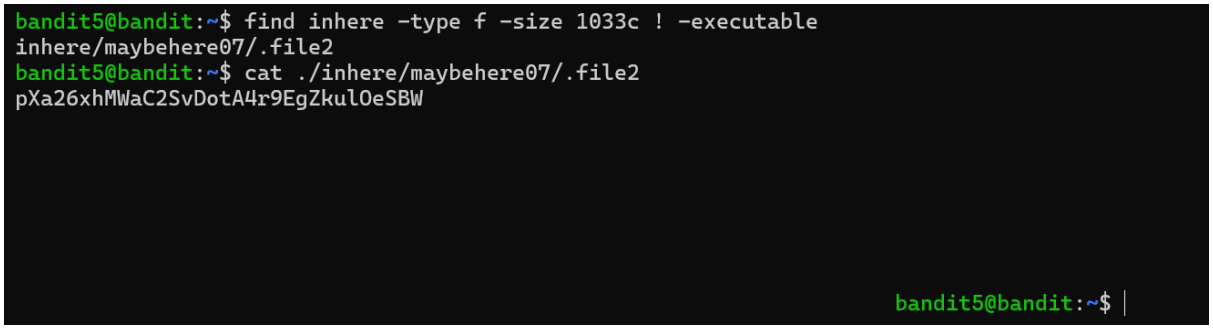
Objective: Find a file in 'inhere' that is human-readable, exactly 1033 bytes, and NOT executable.

Commands used:

```
find inhere -type f -size 1033c ! -executable
cat <the file find returns>
```

Reasoning / Explanation: 'find' searches for files by criteria. '-size 1033c' means exactly 1033 bytes, and '! -executable' excludes executable files. Then read the single match.

Password obtained: pXa26xhMWaC2SvDotA4r9EgZkulOeSBW



Level 6 → Level 7

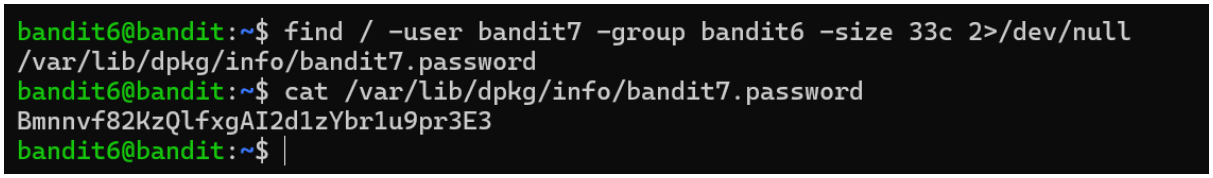
Objective: Find a file located somewhere on the whole server, owned by user bandit7, group bandit6, and 33 bytes in size.

Commands used:

```
find / -user bandit7 -group bandit6 -size 33c 2>/dev/null
cat <the file find returns>
```

Reasoning / Explanation: Searching from '/' scans the entire system. '2>/dev/null' hides the many 'Permission denied' messages so the real result is easy to spot.

Password obtained: Bmnnvf82KzQlfxgAI2d1zYbr1u9pr3E3



Level 7 → Level 8

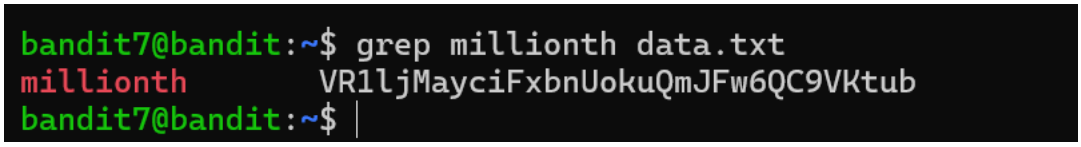
Objective: The password is in data.txt, next to the word 'millionth'.

Commands used:

```
grep millionth data.txt
```

Reasoning / Explanation: 'grep' searches a file for lines containing specific text and prints them. The line containing 'millionth' also contains the password.

Password obtained: VR1ljMayciFxbnUokuQmJFw6QC9VKtub



Level 8 → Level 9

Objective: The password is the only line in data.txt that occurs exactly once.

Commands used:

```
sort data.txt | uniq -u
```

Reasoning / Explanation: 'uniq -u' prints only lines that are not repeated, but it needs its input sorted first — so we pipe 'sort' into it. The single unique line is the password.

Password obtained: EjmOSvuAu7sGAHqHVcBDPirRe9T03kxl

```
bandit8@bandit:~$ sort data.txt | uniq -u
EjmOSvuAu7sGAHqHVcBDPirRe9T03kxl
bandit8@bandit:~$
```

Level 9 → Level 10

Objective: Find human-readable strings in data.txt that are preceded by several '=' characters.

Commands used:

```
strings data.txt | grep =
```

Reasoning / Explanation: data.txt is mostly binary. 'strings' extracts the readable text from it, and 'grep =' filters to the lines marked with '=' signs, where the password sits.

Password obtained: B0s2khmbT9u0geKuOoVGW3JZKhndE3BG

```
bandit9@bandit:~$ strings data.txt | grep =
========== the
==T]
A-=vV
~%9=
========== password
========== is
~4=s\
ediG=N$W
                =['
t       |]=
========== B0s2khmbT9u0geKuOoVGW3JZKhndE3BG
bandit9@bandit:~$
```

Level 10 → Level 11

Objective: data.txt contains the password in base64-encoded form.

Commands used:

```
base64 -d data.txt
```

Reasoning / Explanation: Base64 is an encoding (not encryption) used to represent data as text. The '-d' flag decodes it back to the original, revealing the password.

Password obtained: pYfOY6HwUsDj5rL9UvyhU7MCmv8vN5Ro

```
bandit10@bandit:~$ base64 -d data.txt
The password is pYfOY6HwUsDj5rL9UvyhU7MCmv8vN5Ro
bandit10@bandit:~$
```

Level 11 → Level 12

Objective: data.txt is encoded with ROT13 (every letter rotated 13 places in the alphabet).

Commands used:

```
cat data.txt | tr 'A-Za-z' 'N-ZA-Mn-za-m'
```

Reasoning / Explanation: 'tr' translates one set of characters into another. Mapping A–Z and a–z forward by 13 positions reverses the ROT13 scrambling and reveals the password.

Password obtained: GROozWPO8QyN0mGrjUkID0WCYkZiQxrN

```
bandit11@bandit:~$ cat data.txt | tr 'A-Za-z' 'N-ZA-Mn-za-m'
The password is GROozWPO8QyN0mGrjUkID0WCYkZiQxrN
bandit11@bandit:~$
```

Level 12 → Level 13 (the tricky one — go slow)

Objective: data.txt is a hexdump of a file that has been compressed many times over. You must reverse the hexdump, then repeatedly decompress until plain text appears.

Commands used:

```
mkdir /tmp/sw12 && cp data.txt /tmp/sw12 && cd /tmp/sw12
xxd -r data.txt > data.bin        # reverse the hexdump back to a binary file
file data.bin                      # ask what kind of file it is now
# then, based on what 'file' says, decompress and repeat:
#   gzip  -> mv data.bin data.gz  && gunzip data.gz
#   bzip2 -> mv data.bin data.bz2 && bunzip2 data.bz2
#   tar   -> mv data.bin data.tar && tar xf data.tar
# run 'file' again after each step and keep going (~a dozen rounds) until it says
ASCII text, then cat it
```

Reasoning / Explanation: Each layer was compressed with a different tool. The trick is to repeat the same loop: check the type with 'file', rename the file to the matching extension, decompress it, and check again. When 'file' finally reports ASCII text, 'cat' it to read the password. This is the hardest level — patience wins.

Password obtained: qQYQiHOBPR8zR61qxYqX45quvihF2uzk

```
bandit12@bandit:~$ mkdir /tmp/sw12
bandit12@bandit:~$ cp data.txt /tmp/sw12
bandit12@bandit:~$ cd /tmp/sw12
bandit12@bandit:/tmp/sw12$ xxd -r data.txt > data
bandit12@bandit:/tmp/sw12$ file data
data: gzip compressed data, was "data2.bin", last modified: Sun Oct 25 13:37:03 2026, max compression, from Unix, original size modul
o 2^32 585
bandit12@bandit:/tmp/sw12$ mv data data.gz
bandit12@bandit:/tmp/sw12$ gunzip data.gz
bandit12@bandit:/tmp/sw12$ file data
data: bzip2 compressed data, block size = 900k
bandit12@bandit:/tmp/sw12$ mv data data.bz2
bandit12@bandit:/tmp/sw12$ bunzip2 data.bz2
bandit12@bandit:/tmp/sw12$ file data
data: gzip compressed data, was "data4.bin", last modified: Sun Oct 25 13:37:03 2026, max compression, from Unix, original size modul
o 2^32 20480
```

```
bandit12@bandit:/tmp/sw12$ mv data data.gz
bandit12@bandit:/tmp/sw12$ gunzip data.gz
bandit12@bandit:/tmp/sw12$ file data
data: POSIX tar archive (GNU)
bandit12@bandit:/tmp/sw12$ mv data data.tar
bandit12@bandit:/tmp/sw12$ tar xf data.tar
bandit12@bandit:/tmp/sw12$ ls
data.tar  data.txt  data5.bin
bandit12@bandit:/tmp/sw12$ mv data5.bin data
bandit12@bandit:/tmp/sw12$ file data
data: POSIX tar archive (GNU)
bandit12@bandit:/tmp/sw12$ mv data data.tar
bandit12@bandit:/tmp/sw12$ tar xf data.tar
bandit12@bandit:/tmp/sw12$ ls
data.tar  data.txt  data6.bin
bandit12@bandit:/tmp/sw12$ mv data6.bin data
bandit12@bandit:/tmp/sw12$ file data
data: bzip2 compressed data, block size = 900k
bandit12@bandit:/tmp/sw12$ cat data
2◆h91AY&SYT◆◆◆◆◆◆\@◆W◆◆P#◆t1◆◆LF|◆◆◆
u◆WK◆hz◆ iMM◆◆M4mCCF#&◆dh◆L◆4z◆ʕ◆G◆◆◆◆A◆◆¨1◆i◆◆◆◆◆gĵ◆◆){N◆◆O◆◆◆/ˌ◆Ad◆˚4
^◆Y◆◆◆◆N$+◆◆@bandit12@bandit:/tmp/sw12$ mv data data.bz2
bandit12@bandit:/tmp/sw12$ bunzip2 data.bz2
bandit12@bandit:/tmp/sw12$ file data
data: POSIX tar archive (GNU)
bandit12@bandit:/tmp/sw12$ mv data data.tar
bandit12@bandit:/tmp/sw12$ tar xf data.tar
bandit12@bandit:/tmp/sw12$ ls
data.tar  data.txt  data8.bin
```

```
bandit12@bandit:/tmp/sw12$ mv data data.tar
bandit12@bandit:/tmp/sw12$ tar xf data.tar
bandit12@bandit:/tmp/sw12$ ls
data.tar  data.txt  data8.bin
bandit12@bandit:/tmp/sw12$ mv data8.bin data
bandit12@bandit:/tmp/sw12$ file data
data: gzip compressed data, was "data9.bin", last modified: Sun Oct 25 13:37:03 2026, max compression, from Unix, original size modul
o 2^32 49
bandit12@bandit:/tmp/sw12$ cat data
◆◆jdata9.bin
           ◆HU(H,..◆/JQ◆,V(
                          ◆
                           ◆◆◆w
◆◆
23,◆◆,◆01-,-◆◆p3*◆◆◆qd◆◆1bandit12@bandit:/tmp/sw12$ mv data data.gz
bandit12@bandit:/tmp/sw12$ gunzip data.gz
bandit12@bandit:/tmp/sw12$ file data
data: ASCII text
bandit12@bandit:/tmp/sw12$ cat data
The password is qQYQiHOBPR8zR61qxYqX45quvihF2uzk
bandit12@bandit:/tmp/sw12$
```

Level 13 → Level 14

Objective: You are given a private SSH key (sshkey.private) instead of a password. Use it to log in as bandit14 and read bandit14's password.

Commands used:

```
ssh -i sshkey.private bandit14@localhost -p 2220
cat /etc/bandit_pass/bandit14
```

Reasoning / Explanation: An SSH key is an alternative to a password. '-i' tells SSH which key file to use. Each user's own password is stored at /etc/bandit_pass/<username>, readable only by that user — so once you are bandit14, you can read it.

Password obtained: _____aaWecNkG4FhxJQxz07uiwzVP6bJiYS65_____

```
bandit13@bandit:~$ cat sshkey.private
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAABlwAAAAdzc2gtcn
NhAAAAAwEAAQAAAYEAuCCoxSR6xDCnsm98kqan1x1JF3mfZ6kZa+BoAHetQ/91F4EkTfmE
E+Xs8yVgLhn1YL0TvyVswMzy33OeFJV/KEzef54V8yZo7jFcx+pQlOF6+BFRy+wsyLASV5
XxD/AtafdbVfLZFGTLC53kOFfT0VxokFmnTlIwRyxRJuNAWP9+fkAtbnfqkkixWqg0ZaLr
1fICYam6vb6ilmNfuiGHZyQNHeTkOKZgAaMnQW6bYlRjnkxsNNxk1pj0sT3MUQdLrPCdyh
l8rXUqZ60NZPA6X82/hJNQJR+zVghXrGmTTlU7MAHN7rTdoq4bIQtDShuZZmd5igp5OvVg
tCn4bRM2CFXl8m0P6gNDpR+VYH5f99MwdXyrDUAzuusH7H6CyawRZZF8rFoqONc6HwTJ2h
YJw2uf85ltD3wR8XWzvnBlf2+U9OEmhs01uEYNtNvhq7Gvz/TGI0wR0hEBmsznSns0jShT
cqVsnTMhHaXse3u2D+tbGoC3wOcOyjeX7Skhq63bAAAFiMnVC+zJ1QvsAAAAB3NzaC1yc2
EAAAGBALggqMUkesQwp7JvfJKmp9cdSRd5n2epGWvgaAB3rUP/dReBCk35hBPl7PMlYC4Z
9WC9E78lbMDM8t9znhSVfyhM3n+eFfMmaO4xXMfqUJThevgRUcvsLMiwEleV8Q/wLWn3W1
Xy2RRkywud5DhX09FcaJBZp05SMEcsUSbjQFj/fn5ALW536pJIsVqoNGWi69XyAmGpur2+
opZjX7ohh2ckDR3k5DimYAGjJ0Fum2JUY55MbDTcZNaY9LE9zFEHS6zwncoZfK11KmetDW
TwOl/Nv4STUCUfs1YIV6xpk05VOzABze603aKuGyELQ0obmWZneYoKeTr1YLQp+G0TNghV
5fJtD+oDQ6UflWB+X/fTMHV8qw1AM7rrB+x+gsmsEWWRfKxaKjjXOh8EydoWCcNrn/OZbQ
98EfF1s75wZX9vlPThJobNNbhGDbTb4auxr8/0xiNMEdIRAZrM50p7NI0oU3KlbJ0zIR2l
7Ht7tg/rWxqAt8DnDso3l+0pIaut2wAAAAMBAAEAAAGAEgTUL1LGFtwCFUK2yK05gKIzjH
IRCPZx7+4qj10m3iAqR84PgZj49W+LVDIkqu5MZpaqT4rsjSOhYv+wCSimJH39SjTgxgZM
v36iK0hBcYhtXchoHlIzAcLFUL/yMtKYxyV3UT5uQwIoIq9lbaQerP7jlrjHWDFP2y85k9
oqams6aEWEjKp8kKs/e/U5B3c9qBbCZ+dRyI7W32vDKvZsB0puZC4JrYeOnqpmRY968lD6
3Lty3WtyDNQ0IgI/s/BIK4wmzFM/drZHOLJYjzJssyvjaSf5XydPq0sK+z/0wRUt1ev4+j
2hBuFqunZN0uq2nIz4LSSoclPtTZqu2PhbQgGJj8OaXSH+TkKoLv+sG8dO+rrWtllkDJPX
YsWBmcWfd/Rj0LnkOBSGu+O0ne4IpK+7wAWNoJwpw8MQkLRJkATay0rsGw3Bt7Bf7oqnDA
gPFNfgckMfhWiORhwezgm5zEK50g4cCgwdCH2YQwQluRQHWKKPat9XjXCxSj3N9D1BAAAA
wF2LTVT5VAvkqhJRe4KeT8RhydARmWdt/6YUY+liDnK/Bm4lhpeUrXJ2frX1sGeAty60ZL
lA31Y+kV9xgS4HNOT2aRTCXdfVJ4XmMCsiTxcRejTo92c3EIfD/ttbxkoLTsR/2tjkYje3
kGZ0HketuIdv7EkperlJCcklOLl6Z3i0jwlFcMuZj98b1u/QAAAMEA5JNGvpj8WT4aQuM7
XME/N/m//8/sAb2DMHUWMY0ySSSNRDbM3pJKkja7nUp0CybwTYTrf/agLxIgdQAo71GpK7
ePRmsGu5+mrreNJKicGyER0IqCBhbWyljGDKFMJNGb2gk0Mu9XNZk9y3lRDmPRcfBMekmV
fAjvy5rgAqyf/PsKznyZE12eGosH+CkfvWXdVRXd/hoOdWIZqaC+w8nLso6WLKhptWz8ke
SghZBBAsJW42cyenvyIdIzpVIcJpubAAAAwQDOOCoj/QwJLwXOd75KuUsfgI0Esq9vLSWl
Ds3fQGhnNMEJZfD9B/W8Xa7VClfPHDItcDoXiSgj4dQnS1JeeVBBUHXLULmKLmNLRNQjLD
0HSgyFYTpxl8tO5ZAlFRnGEwe7y9RvNeXp9zPQC2v1FTVQ2RdALiWvR4fGyDu0ER4hYiA/
WfqaBSBABkdhyWLiTBja/VncIHBZ04Bk1S9nwc0z6USyTXpTwpZIJw1O74Tjk+8X3or8WC
kN158egNsB+sEAAAAOcnVkeUBsb2NhbGhvc3QBAgMEBQ==
-----END OPENSSH PRIVATE KEY-----
bandit13@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
PS C:\Users\Swathi Tathireddy> notepad $HOME\sshkey.private
PS C:\Users\Swathi Tathireddy> icacls $HOME\sshkey.private /inheritance:r /grant:r "$($env:USERNAME):R"
processed file: C:\Users\Swathi Tathireddy\sshkey.private
Successfully processed 1 files; Failed processing 0 files
PS C:\Users\Swathi Tathireddy> ssh -i $HOME\sshkey.private bandit14@bandit.labs.overthewire.org -p 2220
```

```
bandit13@bandit:~$ ls
HINT  sshkey.private
bandit13@bandit:~$ ssh -i sshkey.private bandit14@localhost -p 2220
The authenticity of host '[localhost]:2220 ([127.0.0.1]:2220)' can't be established.
ED25519 key fingerprint is: SHA256:C2ihUBV7ihnV1wUXRb4RrEcLfXC5CXlhmAAM/urerLY
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Could not create directory '/home/bandit13/.ssh' (Permission denied).
Failed to add the host to the list of known hosts (/home/bandit13/.ssh/known_hosts).
                         _                     _ _ _
                        | |__   __ _ _ __   __| (_) |_
                        | '_ \ / _` | '_ \ / _` | | __|
                        | |_) | (_| | | | | (_| | | |_
                        |_.__/ \__,_|_| |_|\__,_|_|\__|
```





Level 14 → Level 15

Objective: Submit the current (bandit14) password to port 30000 on localhost to receive the next one.

Commands used:

```
cat /etc/bandit_pass/bandit14      # view the password to send
nc localhost 30000                 # then paste the password and press Enter
```

Reasoning / Explanation: 'nc' (netcat) opens a raw network connection to a port. After you send the correct password, the service replies with the next level's password.

Password obtained: _____pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7_____



Level 15 → Level 16

Objective: Submit the current password to port 30001 — but this port only accepts an SSL/TLS (encrypted) connection.

Commands used:

```
openssl s_client -connect localhost:30001 -quiet
# then paste the bandit15 password and press Enter
```

Reasoning / Explanation: Plain netcat won't work here because the port demands encryption. 'openssl s_client' makes an encrypted TLS connection. Add '-quiet' if the session closes too early.

Password obtained: _____kS0Hf0u5HiXFwKMKFqXvPdOTNGGa0X8V_____

```
bandit15@bandit:~$ openssl s_client -connect localhost:30001 -quiet
Connecting to 127.0.0.1
Can't use SSL_get_servername
depth=0 CN=SnakeOil
verify error:num=18:self-signed certificate
verify return:1
depth=0 CN=SnakeOil
verify return:1
pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7
Correct!
kS0Hf0u5HiXFwKMKFqXvPdOTNGGa0X8V
```

Level 16 → Level 17

Objective: Scan ports 31000–32000, find the one that speaks SSL and returns a key, then use that SSH key to log in as bandit17.

Commands used:

```
nmap -p 31000-32000 localhost        # find which ports are open
openssl s_client -connect localhost:<open port> -quiet   # try the SSL ones,
submit bandit16 password
# one port returns an RSA PRIVATE KEY — copy it into a file:
nano key.private                     # paste the key, save (or use vi)
chmod 600 key.private                # keys must be private to be accepted
ssh -i key.private bandit17@localhost -p 2220
```

Reasoning / Explanation: 'nmap' discovers open ports. You test the SSL ones; the correct port hands back a private key. You save that key, lock down its permissions with 'chmod 600', and use it to log in as bandit17.

Password obtained: _____pWXMAZoxGC8JmDMfmT5MGEsobMM3vnj2_____

```
bandit16@bandit:~$ nmap -p 31000-32000 localhost
Starting Nmap 7.98 ( https://nmap.org ) at 2026-10-27 22:02 +0000
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00013s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 996 closed tcp ports (conn-refused)
PORT      STATE SERVICE
31046/tcp open  unknown
31518/tcp open  unknown
31691/tcp open  unknown
31790/tcp open  unknown
31960/tcp open  unknown

Nmap done: 1 IP address (1 host up) scanned in 0.06 seconds
bandit16@bandit:~$ mkdir /tmp/sw16
bandit16@bandit:~$ echo kS0Hf0u5HiXFwKMKFqXvPdOTNGGa0X8V | openssl s_client -connect localhost:31790 2>/dev/null | sed -n '/BEGIN/,/END/p' > /tmp/sw16/bandit17.key
bandit16@bandit:~$ cat /tmp/sw16/bandit17.key
-----BEGIN CERTIFICATE-----
MIIFBzCCAu+gAwIBAgIUBLz7DBxA0IfojaL/WaJzE6Sbz7cwDQYJKoZIhvcNAQEL
BQAwEzERMA8GA1UEAwwIU25ha2VPaWwwHhcNMjQwNjEwMDM1OTUwWhcNMzQwNjA4
MDM1OTUwWjATMREwDwYDVQQDDAhTbmFrZU9pbDCCAiIwDQYJKoZIhvcNAQEBBQAD
ggIPADCCAgoCggIBANI+P5QXm9Bj21FIPsQqbqZRb5XmSZZJYaam7EIJ16Fxedf+
jXAv4d/FVqiEM4BuSNsNMeBMx2Gq0lAfN33h+RMTjRoMb8yBsZsC063MLfXCk4p+
09gtGP7BS6Iy5XdmfY/fPHvA3JDEScdlDDmd6Lsbdwhv93Q8M6POVO9sv4HuS4t/
jEjr+NhE+Bjr/wDbyg7GL71BP1WPZpQnRE4OzoSrt5+bZVLvODWUFwinB0fLaGRk
GmI0r5EUOUd7HpYyoIQbiNlePGfPpHRKnmdXTTEZEoxeWWAaM1VhPGqfrB/Pnca+
vAJX7iBOb3kHinmfVOScsG/YAUR94wSELeY+UlEWJaELVUntrJ5HeRDiTChiVQ++
wnnjNbepaW6shopybUF3XXfhIb4NvwLWpvoKFXVtcVjlOujF0snVvpE+MRT0wacy
tHtjZs7Ao7GYxDz6H8AdBLKJW67uQon37a4MI260ADFMS+2vEAbNSFP+f6ii5mrB
18cY6l4ZaF6oU8bjGK7BArDx56bRc3WFyuBIGWAFHEuB948BcshXY7baf5jjzPmgz
mq1zdRthQB31MOM2ii6vuTkheAvKfFf+llH4M9SnES4NSF2hj9NnHga9V08wfhYc
x0W6qu+S8HUdVF+V23yTvUNgz4Q+UoGs4sHSDEsIBFqNvInnpUmtNgcR2L5PAgMB
AAGjUzBRMB0GA1UdDgQWBBTPo8kfze4P9EgxNuyk7+xDGFtAYzAfBgNVHSMEGDAW
gBTPo8kfze4P9EgxNuyk7+xDGFtAYzAPBgNVHRMBAf8EBTADAQH/MA0GCSqGSIb3
DQEBCwUAA4ICAQAKHomtmcGqyiLnhziLe97Mq2+Sul5QgYVwfx/KYOXxv2T8ZmcR
Ae9XFhZT4jsAOUDK1OXx9aZgDGJHJLNEVTe9zWv1ONFfNxEBxQgP7hhmDBWdtj6d
taqEW/Jp06X+08BtnYK9NZsvDg2YRcvOHConeMjwvEL7tQK0m+GVyQfLYg6jnrhx
egH+abucTKxabFcWSE+Vk0uJYMqcbXvB4WNKz9vj4V5Hn7/DN4xIjFko+nREw6Oa
/AUFjNnO/FPjap+d68H1LdzMH3PSs+yjGid+6Zx9FCnt9qZydW13Miqg3nDnODXw
+Z682mQFjVlGPCA5ZOQbyMkY4tNazG2n8qy2famQT3+jF8Lb6a4NGbnpeWnLMkIu
jWLWIkA9MlbdNXuajiPNVyYIK9gdoBzbfaKwoOfSsLxEqlf8rio1GGcEV5HlzS52
txwI0xdW9MWeGWoiLbZSbRJH4TIBFFtoBG0L0EJi0C+UPwS8CDngJB4TyrZqEld3
rH87W+Et1t/Nepoc/Eoaux9PFp5VPXP+qwQGmhir/hv7OsgBhrkYuhkjxZ8+luk7
tUWC/XM0mpLoxsq6vVl3AJaJelivdA9xLytsuG4iv02Juc593HXYR8yOpow0Eq2T
U5EyeuFg5RXYwAPi7ykw1PW7zAPL4MlonEVz+QXOSx6eyhimp1VZC11SCg==
-----END CERTIFICATE-----
```

```
bandit16@bandit:~$ echo kS0Hf0u5HiXFwKMKFqXvPdOTNGGa0X8V | timeout 3 openssl s_client -connect localhost:31790 -quiet 2>/dev/null | sed -n '/BEGIN.*PRIVATE KEY/,/END.*PRIVATE KEY/p' > /tmp/sw16/bandit17.key
bandit16@bandit:~$ cat /tmp/sw16/bandit17.key
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAABlwAAAAdzc2gtcn
NhAAAAAwEAAQAAAYEAvdSaw8j1FQ2DjtbQPGiEVtqEG5kt3g71uDlixg42vRN2MvWRVnGQ
t4k9T9tDWaisnn+6I4RCkhEzw231WA6KVc0Sd0+6/6Cp1Egp4o4l+xf5gPNo7A2OqjqN67
Hhy6I71GBjyUBnp6vEtkI3WZmZtuxpCMPyHSy7m56lipJFddKEOUCX21hNWWy2SAZQFBub
3M1hrcar5cA4pCFJ2AmjSsOP4yRbdERh3vZTGNjKe2x+ze4jf2/Y/uNdmixdaAMuD8to4Y
f7JylXL/+ohzasOYM0iNFvr8gkOOc11xuTNdbGNmu1Ff3Vp1qtJNB600EWrBt9H4xl7/WX
wEQ0/3EbpjUxGm3ZyUU5FmD4CGh1l9w4FqMD+RT9T3AVuzX8NM1FiIAkQMe0b34qF7iTjd
Tc+2Ve7Ywaakm79JYFnwirYd9QORxmjqUO+H6Yn9xLFmpRkFjvV+f3NfvekRtV5Fm7le9wr
ipXljZ1hkHfH6echM3pINiJJHiZAgB/CDPVRdLhtAAAFiPHONUjxzjVIAAAAB3NzaC1yc2
EAAAGBAL3UmsPI9RUNg47W0DxohFbahBuZLd4O9bg5YsYONr0TdjL1kVZxkLeJPU/bQlmo
rJ5/uiOEQpIRM8Nt9VgOilXNEndPuv+gqdRIKeKOJfsX+YDzaOwNjqo6jeux4cuiO9RgY8
lAZ6erxLZCN1mZmbbsaQjD8h0su5uepYqSRXXShDlAl9tYTVlstkgGUBQbm9zNYa3Gq+XA
OKQhSdgJo0rDj+MkW3REYd72UxjYyntsfs3uI39v2P7jXZosXWgDLg/LaOGH+ycpVy//qI
c2rDmDNIjRb6/IJDjnNdcbkzXWxjZrtRX91adarSTQetNBFqwbfR+MZe/1l8BENP9xG6Y1
MRpt2clFORZg+AhodZfcOBajA/kU/U9wFbs1/DTNRYiAJEDHtG9+Khe4k43U3PtlXu2MGm
pJu/SWBZ8Iq2H4fUDkcZo6lDvh+mJ/cSxZqUZBY71X9zX73pEbVeRZu5XvcK4qV5Y2dYZB3
x+nnITN6SDYiSR4mQIAfwgz1UXS4bQAAAAMBAAEAAAGACMy4N+cy5TzxIkf28zXtHJGYmi
bpp2eOIHIYk8HMm8sxKX+UsyskiD2GaBND9f4Jsnc9S7Qv2dGOUrrgKqrR4tRUzM8XXg42
kS6fMm9gdlPKZke/gJK4L1CIvDmBKiKmXe2aHfh1jXyMnizVCX4qDAhVlSu/oc6UyZxih
Dpw2J02qqR34siWsjdUk1onOYCvaOPqZySD15vwbwBTlB0D10taFwhGSyqVMmaZIZ4LGyF
HEqzvo6Swo4Lor/3vICZJ5YLuUVa2GEEx5Ir1Np/fb3C+zKe37+HPf5lhDps2OWXNf1D/N
KhPt9QbhANoATORB+64nNw66/515vslh87JMn4Yy/mJjJe0uR8cC4nnqXGBOy6lIFzbNQN
DastUidaMaqpswS49R5/Uq2YYOjbU+YCbBJz8qaz8eUMhlMsOI6b2XGwtr4rP9fENWrqxs
z3bYvw2I4t8G/OgZESZvn+DCTAuc/+/NtIeLDTeJJsUggkU5Xm4Xdmz1y0SwRqTRpJAAAA
wQCiE/31KZCUQJfwdZ1Ll6iXZ9ANreda++OlCkVQTGmfjnPAwpc2io/n0IkjE5Rch9bHkR
n/Pnm228x2TaWcq0FsyP9VnZQIw3LYPZxxouvV4ODFeThi6dJij9X7WnyvNVaeQam5Mqzd
6eI4L9f6p43JivvRLc7IrEDMjSXMcnlUbvEFa/143fpHZer9q+9qARUSLIodr8D6zde3l0
r88E0Z0YZrWn1BzjPZr2z+3GPTcfYPM+pLPT3OgAjd7gVr7pEAAADBAN2qsjh6rfgKHiou
n+pf1TUIXLzpnY+icwYcotvfhjweF1KwowzqnNjG0olJqc5B6O2g8FbeIn3a1v/896Ynb3
WXXYslcCXGyyWxkw5nWaSWS8GMVEpjIgvW46hnrWmDVEPuW84wsgZ1yGnL0InHq3SmGMVe
7FLVoO2LD393RW/2RcMZ8mX/SWGLst9IunzxoEHGxJObKWv6C2IgQj8zHDpuE/6TwdDeFS
3KWM+JyggnB+EEssW7Tu+N2H+3mgLNbwAAAMEA2zuReO3x3LioX2USO2ZmawKeajDKAUWh
OmfbD3ab8psuVcllydLWQfmJmJ7xXyAEtmO2kIg6ax6AEd4PLAgDC504v+bmLPjdvSwqGk
//vONxwDY+Uy3m3oX+MHK2KRq5Zd3YJd9Px6AF5iMbyiQYA69nsBumqt04Ihe8CFYHa9uG
KLElQobuX5Wx6cWaOsc1j6lvpaYDEwMUT8LeMFqKjN1rF1LMiNENBQhtd+ikJmYYwB01/5
Pfos/2C+rbNuHjAAAADnJ1ZHlAbG9jYWxob3N0AQIDBA==
-----END OPENSSH PRIVATE KEY-----
bandit16@bandit:~$ exit
```

```
PS C:\Users\Swathi Tathireddy> icacls $HOME\bandit17.key /inheritance:r /grant:r "$($env:USERNAME):R"
processed file: C:\Users\Swathi Tathireddy\bandit17.key
Successfully processed 1 files; Failed processing 0 files
PS C:\Users\Swathi Tathireddy> ssh -i $HOME\bandit17.key bandit17@bandit.labs.overthewire.org -p 2220
                         _                     _ _ _
                        | |__   __ _ _ __   __| (_) |_
                        | '_ \ / _` | '_ \ / _` | | __|
                        | |_) | (_| | | | | (_| | | |_
                        |_.__/ \__,_|_| |_|\__,_|_|\__|


                      This is an OverTheWire game server.
            More information on http://www.overthewire.org/wargames
```

```
bandit17@bandit:~$ cat /etc/bandit_pass/bandit17
pWXMAZoxGC8JmDMfmT5MGEsobMM3vnj2
bandit17@bandit:~$
```

Level 17 → Level 18

Objective: Two files, passwords.old and passwords.new, are almost identical. The password is the single line that changed.

Commands used:

```
diff passwords.old passwords.new
```

Reasoning / Explanation: 'diff' compares two files line by line and shows only what differs. The changed line (marked with '>') in passwords.new is the password.

Password obtained: _____OQxXZjELndr90zuhOTDYBEomI0SZITXI_____

```
bandit17@bandit:~$ cat /etc/bandit_pass/bandit17
pWXMAZoxGC8JmDMfmT5MGEsobMM3vnj2
bandit17@bandit:~$ ls
passwords.new  passwords.old
bandit17@bandit:~$ diff passwords.old passwords.new
42c42
< 1WJHmb0k7TH4piWp9Pd0lsruErLQZrR9
---
> OQxXZjELndr90zuhOTDYBEomI0SZITXI
bandit17@bandit:~$
```

Level 18 → Level 19

Objective: The password is in 'readme' in the home directory, but .bashrc has been rigged to log you out the instant you connect.

Commands used:

```
ssh bandit18@bandit.labs.overthewire.org -p 2220 cat readme
```

Reasoning / Explanation: By adding a command ('cat readme') to the end of the ssh line, that command runs on the remote server BEFORE the auto-logout can kick in — so you still get the file's contents.

Password obtained: _____KpsOfPkcP7i1FlIExk2QEjyt6dw8dxZI_____



Level 19 → Level 20

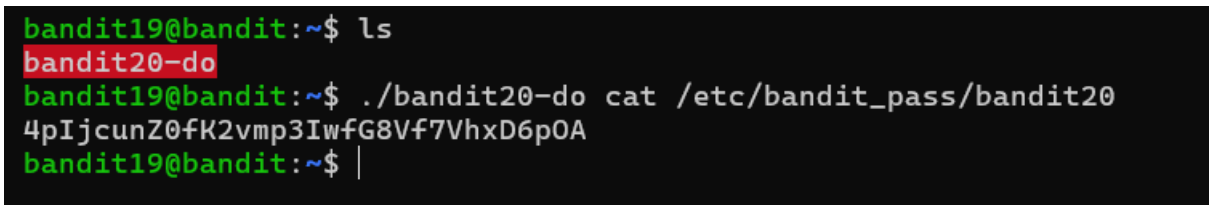
Objective: Use the setuid helper binary 'bandit20-do' to read bandit20's password file.

Commands used:

```
./bandit20-do cat /etc/bandit_pass/bandit20
```

Reasoning / Explanation: A 'setuid' program runs with the permissions of another user. 'bandit20-do' lets you run one command as bandit20, so you can read bandit20's otherwise-protected password file.

Password obtained: _____4pIjcunZ0fK2vmp3IwfG8Vf7VhxD6pOA_____



Level 20 → Level 21 (needs two terminals open)

Objective: The 'suconnect' binary connects to a port you choose; if you send it the CURRENT password first, it sends back the NEXT one.

Commands used:

```
# Terminal A (a second SSH session) — start a listener that sends the current
password:
echo <bandit20 password> | nc -lvp 55555

# Terminal B (your first session) — tell suconnect to connect to that port:
./suconnect 55555
```

Reasoning / Explanation: You open two SSH sessions. In one you run a listener (nc -lvp) holding the current password. In the other you run suconnect, which connects to your listener, receives the password, verifies it, and replies with bandit21's password. Pick any free port number (e.g. 55555).

Password obtained: _____bW9kBv5WC3P4yoDyf12LSdGuNz5ka6hY_____

```
bandit19@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
PS C:\Users\Swathi Tathireddy> ssh bandit20@bandit.labs.overthewire.org -p 2220
                         _                     _ _ _
                        | |__   __ _ _ __   __| (_) |_
                        | '_ \ / _` | '_ \ / _` | | __|
                        | |_) | (_| | | | | (_| | | |_
                        |_.__/ \__,_|_| |_|\__,_|_|\__|
```

```
bandit20@bandit:~$ echo 4pIjcunZ0fK2vmp3IwfG8Vf7VhxD6pOA | nc -lvp 55555
Listening on 0.0.0.0 55555
Connection received on localhost 54226
bW9kBv5WC3P4yoDyf12LSdGuNz5ka6hY
bandit20@bandit:~$
```

```
bandit20@bandit:~$ ./suconnect 55555
Read: 4pIjcunZ0fK2vmp3IwfG8Vf7VhxD6pOA
Password matches, sending next password
bandit20@bandit:~$
```

4. Conclusion

Completing Bandit Levels 0 to 20 built a practical foundation in Linux command-line navigation, file inspection, searching and filtering text, basic encoding/decoding (base64, ROT13), working with compressed files, SSH key authentication, raw and encrypted network connections (netcat and openssl), port scanning with nmap, and privilege concepts such as setuid binaries. Each level reinforced a core skill that maps directly to real security work: reconnaissance, careful observation of output, and reasoning about how systems protect and expose information.

Title: What I Learned Hacking Through OverTheWire Bandit (Levels 0–20)

When I started the OverTheWire Bandit challenge, I'll be honest — the terminal felt intimidating. A black screen, a blinking cursor, and no buttons to click. But by the time I reached Level 20, that same terminal had started to feel like home. If you're a student just getting into cybersecurity, Bandit is honestly one of the best places to begin, and I want to share why.

Bandit is a free "wargame." You log into a remote server using SSH, and each level is a small puzzle. Solving it gives you a password that unlocks the next level. That's the whole loop, but inside that loop, you quietly learn a huge amount of real Linux and security skills without even noticing.

The early levels teach you the basics: listing files, reading them, dealing with awkward filenames that have spaces or dashes, and finding hidden files. It sounds simple, but these are the exact commands security analysts use every day. Then things get more interesting. You learn to search with grep, filter unique lines with sort and uniq, pull readable text out of binary files with strings, and decode data with base64 and ROT13.

The level that nearly broke me was Level 12. The file had been compressed over and over gzip inside bzip2 inside tar, layer after layer. The trick was to stay calm and repeat one simple loop: check the file type, rename it, decompress it, and check again. It taught me something bigger than a command patience. In security, the answer is often just one more careful step away.

The later levels moved into networking: connecting to ports with netcat, doing the same thing but encrypted with openssl, scanning for open ports with nmap, and using SSH private keys instead of passwords. The final level even needed two terminals working together — one listening, one connecting. Pulling that off felt genuinely satisfying.

Here are a few things I wish someone had told me on day one:

- **Your password won't show when you type it.** That's normal, not a bug. Just type it and press Enter.
- **You can't jump from one level straight to the next inside the same window.** Log out (exit) first, then log back in.
- **Read the error messages.** More than once, the terminal literally told me how to fix the problem.
- **Keep notes of every password.** You'll need the previous one to get the next.
- **Don't rush the hard levels.** Break them into small, repeatable steps.

The biggest lesson wasn't any single command — it was learning *how to think*. How to look at a problem, notice the small clues, and work through it one piece at a time. That mindset is what security is really about.

If you're nervous about starting, don't be. Open a terminal, type that first ssh command, and just begin. Bandit meets you where you are and grows with you. By the end, you won't just know some Linux commands — you'll feel like you can actually poke at a system and figure it out. And that feeling is worth every confusing moment along the way.